Module 2: Training-Loop Anatomy

Every State Change Behind One Optimizer Update

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. Optimizer 101
2. Base loop
3. Additional training features
4. FLOP and memory accounting

Guiding question

Which state changes at each line of a training loop, and what silently breaks when the order changes?

What should be clear by the end?

1. Explain SGD, learning rate, gradients, and optimizer state.
2. Trace the base loop from activations and loss to a parameter update.
3. Place accumulation, gradient clipping, and scheduling around an update.
4. Estimate training FLOPs and compare SGD and Adam memory.

PART 1

Optimizer 101

1. Optimizer 101
2. Base loop
3. Additional training features
4. FLOP and memory accounting

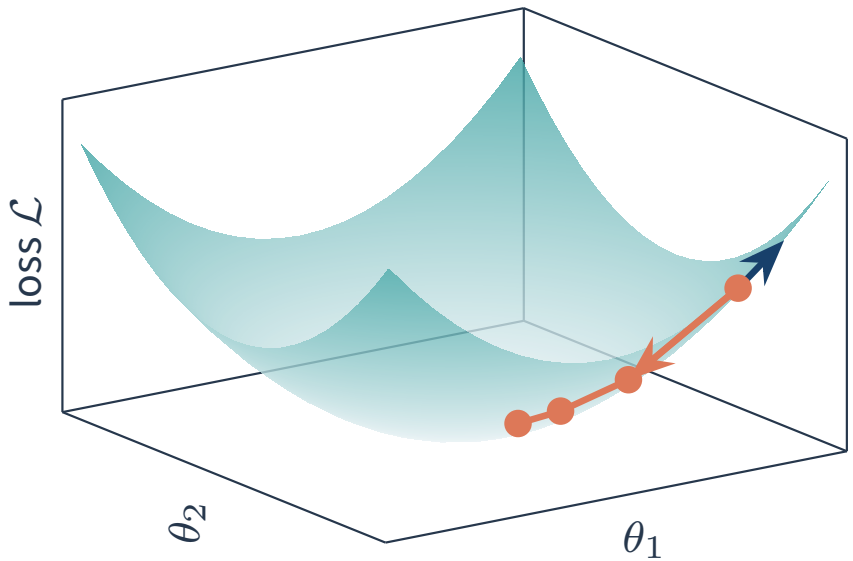
From predictions to an update

1. The **forward pass** transforms tokens through activations into logits.
2. The **loss** compares predictions with next-token targets.
3. **Backward** uses saved activations and the chain rule to compute $g_t = \nabla_\theta \mathcal{L}_t$.
4. The **optimizer** reads gradients and updates parameters (and, for some optimizers, its own state).

Different lifetimes

Saved activations serve backward; gradients are cleared between updates. Weights and optimizer state persist across updates.

A loss landscape: follow the downhill direction



Schematic: two parameters and one fixed loss surface.

Gradient

$g_t = \nabla_\theta \mathcal{L}(\theta_t)$ points locally uphill.

Descent step

Move against the gradient:

$$\theta_{t+1} = \theta_t - \eta g_t$$

Learning rate η scales the step:

→ small η ⟶ large η

Too large can overshoot.

Gradient descent and SGD

$$\theta_{t+1} = \theta_t - \eta_t \nabla_\theta \mathcal{L}_t(\theta_t)$$

- The gradient gives the local direction of increasing loss; subtract it to move downhill.
- The **learning rate** η_t sets the step size. Too large can be unstable; too small can be slow.
- **Stochastic gradient descent (SGD)** estimates the gradient from a sampled example or mini-batch, instead of the full dataset.
- Plain SGD has no per-parameter optimizer state; momentum SGD adds a history buffer.

Adam: gradients build optimizer state

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- Adam stores two running moments per parameter; each step updates them from g_t.
- Activations and loss produce gradients through backward; they are not optimizer state.
- AdamW also applies decoupled weight decay. Save optimizer state to resume its trajectory.

PART 2

Base loop

1. Optimizer 101
2. Base loop
3. Additional training features
4. FLOP and memory accounting

Five lines hide four kinds of state

1. clear parameter gradient buffers
2. build a forward computation graph
3. accumulate derivatives into `.grad`
4. update parameters and optimizer moments

Minimal loop

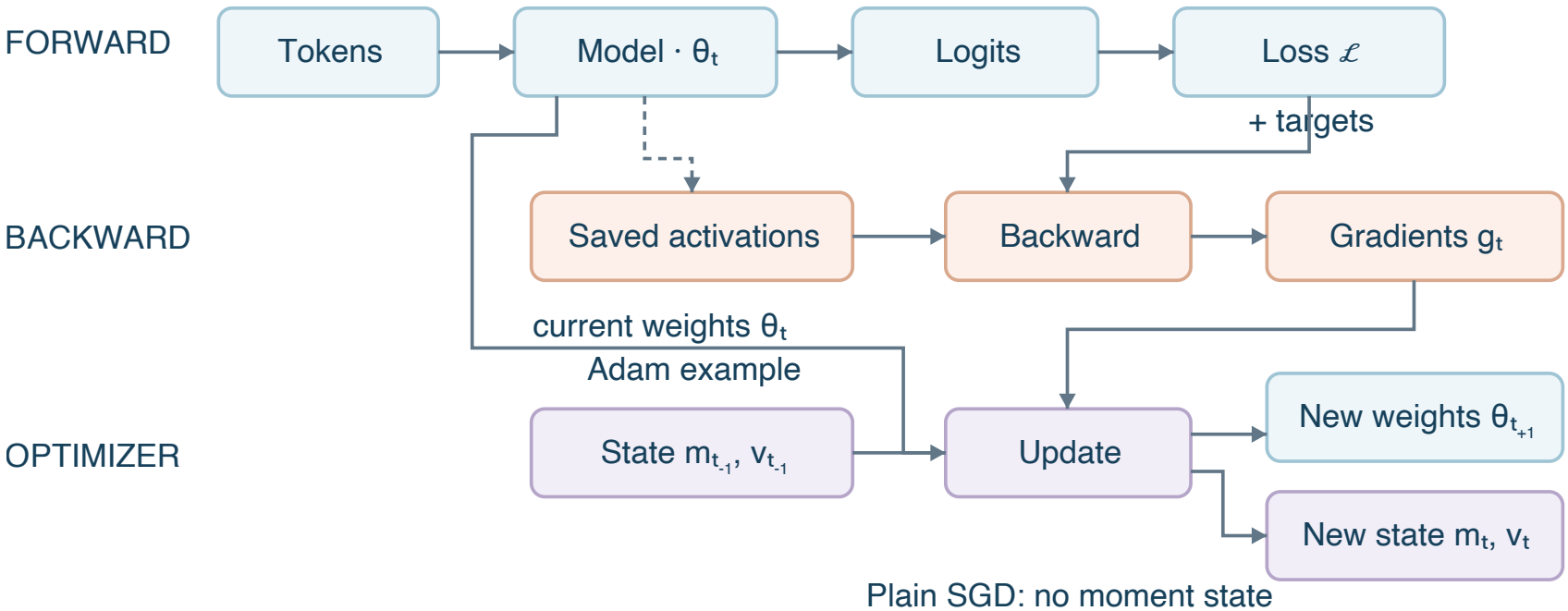
```
zero_grad()
logits, loss =
model(x,y)
loss.backward()
optimizer.step()
```

Crucial distinction

`backward()` computes gradients. Only `optimizer.step()` changes weights.

Source: The Annotated PyTorch Training Loop

From forward to optimizer state



Saved activations are used for backward; optimizer state persists across updates.

Source: Inspired by the Ultra-Scale Playbook

Backward accumulates into persistent buffers

For each parameter θ_i, autograd adds the current derivative:

$$\theta_i.\text{grad} \; += \; \frac{\partial \mathcal{L}}{\partial \theta_i}$$

This behavior enables gradient accumulation, but it also makes a missing reset dangerous.

Why `set_to_none=True`?

Deallocate buffers rather than write zeros across every element. The next backward pass recreates them.

Diagnostic

Log whether gradients are `None` before the first backward pass.

PART 3

Additional training features

1. Optimizer 101
2. Base loop
3. Additional training features
4. FLOP and memory accounting

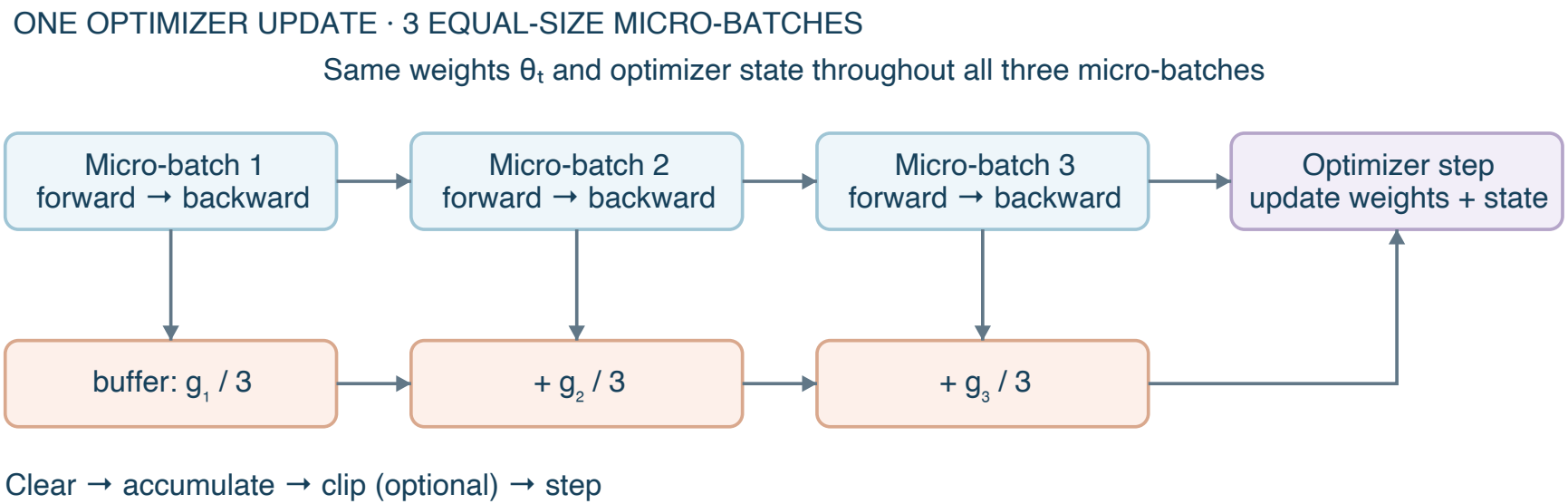
Three additions around the same update

1. **Accumulation:** combine gradients from several micro-batches when the desired batch does not fit at once.
2. **Clipping:** limit the global gradient norm before an unusually large update.
3. **Scheduler:** vary the learning rate over optimizer updates.

One update boundary

Accumulate backward passes $\rightarrow$ clip $\rightarrow$ optimizer step $\rightarrow$ advance schedule $\rightarrow$ clear gradients.

Accumulation: three micro-batches, one update



For equal-size micro-batches, divide each mean loss by three before backward.

Source: Inspired by the Ultra-Scale Playbook

Gradient accumulation changes update frequency

$$N_{\text{tokens/update}} = B_{\text{micro}}\, T\, K\, W$$

- K micro-batches contribute to one update.
- Divide each equal-size micro-loss by K.
- Clip and step only at the update boundary.

Worked count

$B_{\text{micro}} = 4, T = 1024, K = 8, W = 2$

$N = 65{,}536$ tokens per optimizer update

Unequal micro-batches

Normalize by the number of valid tokens, not blindly by the number of micro-steps.

Global clipping preserves direction

$$\|g\|_2 = \sqrt{\sum_i \|g_i\|_2^2}$$

$$g_i \leftarrow g_i \cdot \min\left(1, \frac{M}{\|g\|_2 + \epsilon}\right)$$

- All parameter gradients receive the same multiplier.
- Log the pre-clip norm and whether clipping activated.
- Frequent saturation is evidence of instability, not proof that clipping fixed it.

The scheduler counts optimizer updates

$$\eta_t = \begin{cases} \eta_{\max} \frac{t+1}{T_w+1} & t < T_w \\ \eta_{\min} + \frac{1+\cos(\pi r)}{2}(\eta_{\max} - \eta_{\min}) & t \geq T_w \end{cases}$$

where $r = (t - T_w)/(T_{\max} - T_w)$.

Warmup

Limits early updates while activations and optimizer moments stabilize.

Silent failure

Stepping once per micro-batch compresses the schedule by factor K.

PART 4

FLOP and memory accounting

---

1. Optimizer 101
2. Base loop
3. Additional training features
4. FLOP and memory accounting

Count the model and optimizer separately

Let N be unique parameters and $D = BT$ processed tokens. A multiply and an add count as two FLOPs.

Dense model work

Matrix forward: $\approx 2Dd_{\text{in}}d_{\text{out}}$ FLOPs. Backward: roughly twice as much. A rough Transformer estimate is $6ND$ for forward + backward.

Optimizer work: once per update

Plain SGD: $\approx 2N$ FLOPs. Adam: $\approx 12N$ using a simple count, omitting bias correction and counting sqrt and division as one each.

The model estimate simplifies attention, embeddings and other operations. FLOPs are not a runtime measurement.

Worked compute estimate

124M parameters, 1M tokens

Forward + backward: $6ND \approx 7.44 \times 10^{14}$ FLOPs.

One SGD update: $2N \approx 2.48 \times 10^{8}$ FLOPs.

One Adam update: $12N \approx 1.49 \times 10^{9}$ FLOPs.

Compare like with like: the model count covers one million tokens; each optimizer count covers one update. Multiply optimizer cost by the number of updates for a run.

Memory: SGD versus Adam

Assume N unique parameters, FP32 (4 bytes per scalar), gradients for all parameters, and SGD without momentum.

Storage	SGD	Adam
Parameters	$4N$ bytes	$4N$ bytes
Gradients	$4N$	$4N$
First and second moments	0	$8N$
Subtotal	$8N$	$16N$

For $N = 124\text{M}$: **0.992 GB** (SGD) versus **1.984 GB** (Adam), decimal GB.

Not peak memory

Add saved activations, temporary buffers, inputs, allocator overhead and other state. Activation memory depends on micro-batch size, context and implementation.

Checkpoint: estimate a run

Think, then compare

A model has $N = 100$M parameters and processes 500,000 tokens in 100 updates. Estimate $6ND$ model FLOPs, total SGD and Adam optimizer FLOPs, and their FP32 parameter/gradient/state memory subtotals. What is missing from peak memory?

Estimate: one possible answer

- Model: $6ND = 3 \times 10^{14}$ FLOPs.
- 100 optimizer updates: SGD 2×10^{10} FLOPs; Adam $\approx 1.2 \times 10^{11}$ FLOPs under our simplified convention.
- FP32 subtotal: SGD 0.8 GB; Adam 1.6 GB, decimal units.
- Peak memory also depends on activations, temporary buffers, inputs and allocation overhead.

From a fixed-batch test to a baseline

First: can the loop learn?

Overfit one fixed shifted batch. Falling loss tests the training path, not generalization.

Then: a controlled run

Use disjoint train and validation documents. Record data, model, optimizer, schedule, seed, hardware, code and token budget.

This afternoon: instructor-prepared JSONL documents; the baseline script packs its own byte-token shards. The data-pipeline lab follows tomorrow.

The afternoon lab: one update, then a baseline

Session 7: make it learn

Shift inputs and targets; implement the FP32 update; overfit one batch; check a checkpoint round-trip.

Session 8: measure a baseline

Run disjoint documents, inspect held-out loss and token count, and test interruption and recovery.

Recovery contract

Resuming needs optimizer, RNG and run identity as well as weights. Use the supplied reference checkpoint code if your extension is unfinished.

Mixed precision is introduced with GPU execution in Module 4.

Read the baseline as evidence

- Count tokens: $B_{\text{micro}} \times T \times K \times$ updates (one device).
- Compare train loss with held-out loss on a fixed validation protocol; a tiny or synthetic corpus is a workflow check, not evidence of language quality.
- Track learning rate and gradient norm to diagnose a spike. For nearly uniform logits, initial cross-entropy is roughly $\log V$.
- Record checkpoints and compare runs at the same token budget. Fixed-prompt samples, if available, are qualitative diagnostics.

Recovery requires more than weights

A resumable training run

Save model weights, optimizer moments, completed step and schedule configuration, RNG state, and the data/configuration identity.

Test the claim

Compare uninterrupted and save/resume runs with the same planned schedule. The companion tests exact CPU continuation; CUDA requires its own verification.

If real documents are unavailable

Run the synthetic smoke fixture and label the result as a mechanics check.

Key takeaways

1. The loss and saved activations produce gradients; the optimizer uses them to update weights and any persistent state.
2. The base loop clears gradients, runs forward and backward, then updates parameters.
3. Accumulation, clipping and scheduling operate around the optimizer update boundary.
4. State assumptions in FLOP and memory estimates; compare baselines using held-out loss, token budgets and recovery checks.